

Greedy Algorithms

CSC 349 - Algorithms

Cal Poly, San Luis Obispo

Rodrigo Canaan

Adapted from materials by Alexander Dekthyar, Theresa Miggler

2 - Greedy algorithms in Graphs

Dijkstra's algorithm
Minimum Spanning Tree

Dijkstra's Algorithm

General Tree Search Algorithm (best-first search)

1. **Initialize fringe (or frontier) with node containing the initial state**
 - **Fringe:** data structure with nodes that we want to explore next
 - Initially contains a node corresponding to the start state
2. **Repeat while you have computational budget:**
 - A. Check if fringe is empty
 - If so, return
 - B. (If not) Select a node **n** in the fringe and remove it from the fringe
 - C. Check if **n** contains a goal state
 - If so, go to 3 (return solution)
 - D. (If not) Expand **n**, adding nodes containing all its successor states to the fringe
3. **Return the action that leads from the initial node to the second node on the path to the goal (online problems) or the entire path (offline)**

The order in which elements are added (or removed) to the fringe defines the search algorithm!

General Tree Search Algorithm (best-first search)

1. **Initialize fringe (or frontier) with node containing the initial state**
 - **Fringe:** data structure with nodes that we want to explore next
 - Initially contains a node corresponding to the start state
2. **Repeat while you have computational budget:**
 - A. Check if fringe is empty
 - If so, return failure
 - B. (If not) Select a node **n** in the fringe and remove it from the fringe
 - C. Check if **n** contains a goal state
 - If so, go to 3 (return solution)
 - D. (If not) Expand **n**, adding nodes containing all its successor states to the fringe
3. **Return the action that leads from the initial node to the second node on the path to the goal (online problems) or the entire path (offline)**

The algorithm is greedy with respect to that order

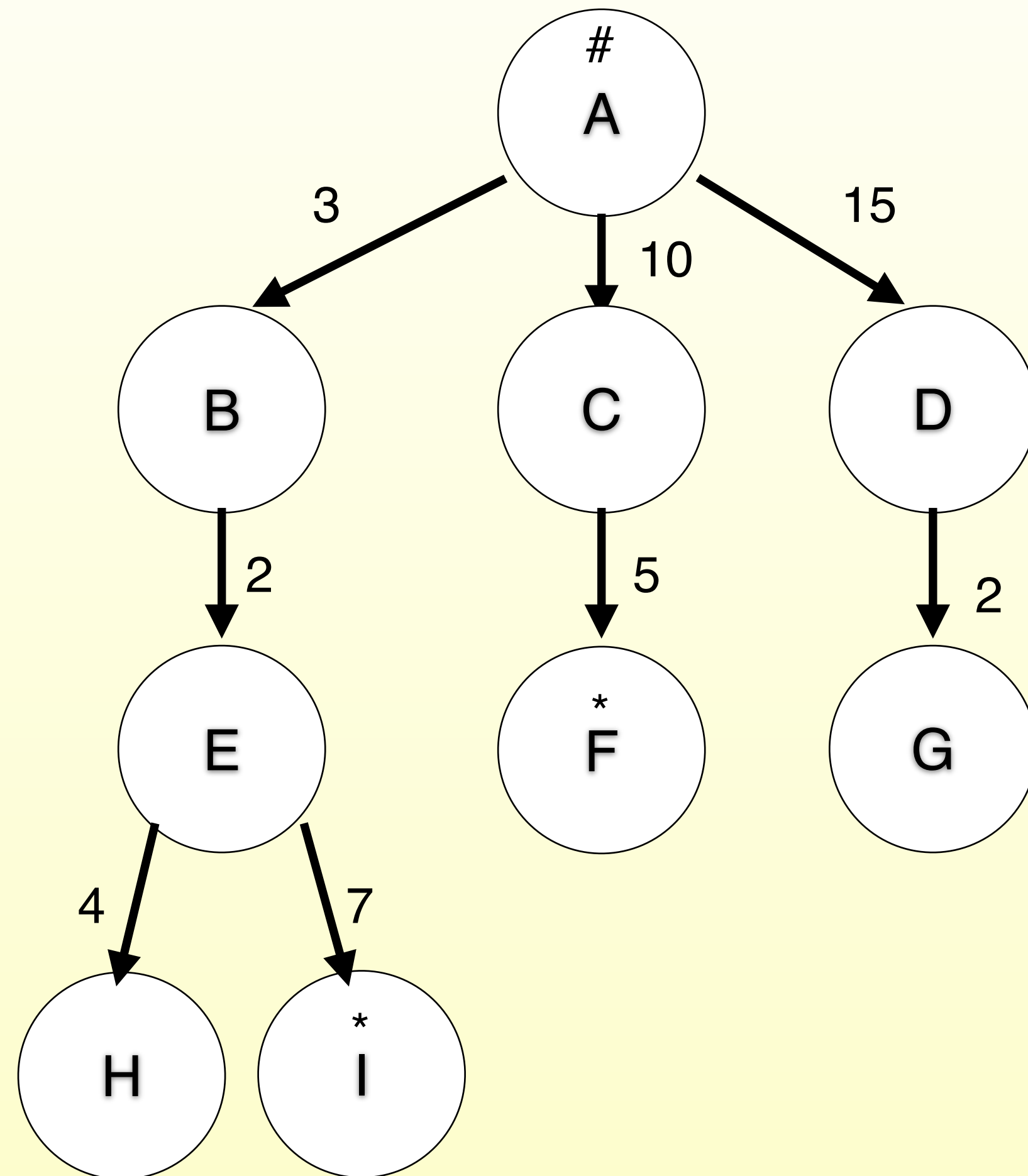
Greedy Heuristics in graph search

- In Depth-First Search: most recent node first
- In Breadth-First Search: oldest node first
 - Node closest to the root first
- What if we have a graph with weighted edges?
 - Same heuristic still works: nodes with a shortest path from the root first
 - But now the length of a path is not just the number of edges, but the sum of their weights

Uniform Cost Search

- A variation of Breadth-First Search that handles edges with variable cost
- Strategy: associate a cost **$g(n)$** to each node n added to the fringe, representing the *total cost of the computed path up to n*
- The algorithm is greedy with respect with **g** - take the node with smallest g to expand next
- Done naively, this may require sorting the fringe each time, which is expensive.
- **Priority Queues** are an abstract data type that allow for quickly finding the smallest element.
 - Heaps are the most common implementation of priority queues
 - See the distinction between abstract and concrete data types (or data structures) at https://en.wikipedia.org/wiki/Abstract_data_type (access 2025-02-13)

Example State Space



Legend:

- Starting state

* - Goal state

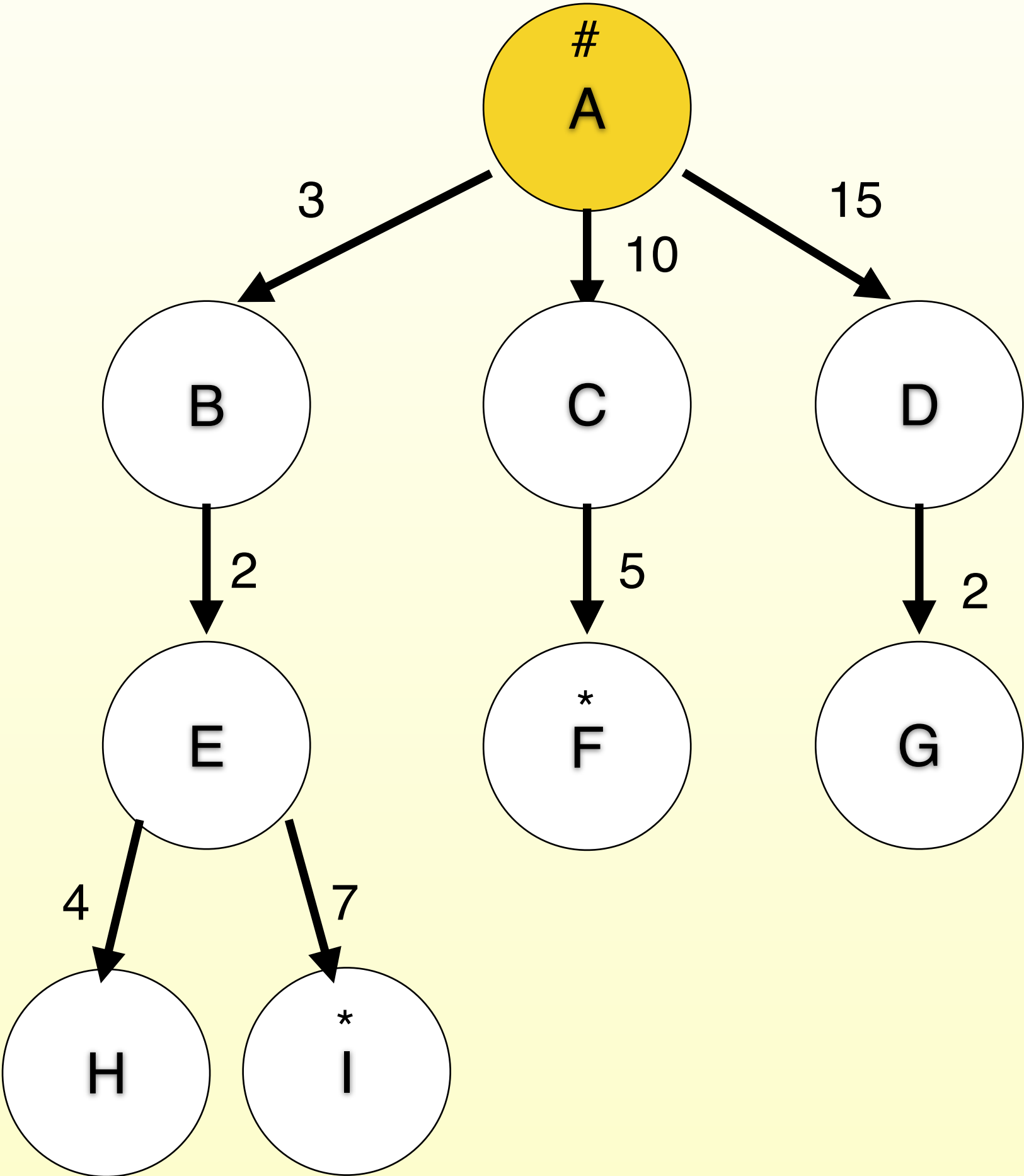
● Visited (expanded)

● Current state

● Fringe (priority queue)

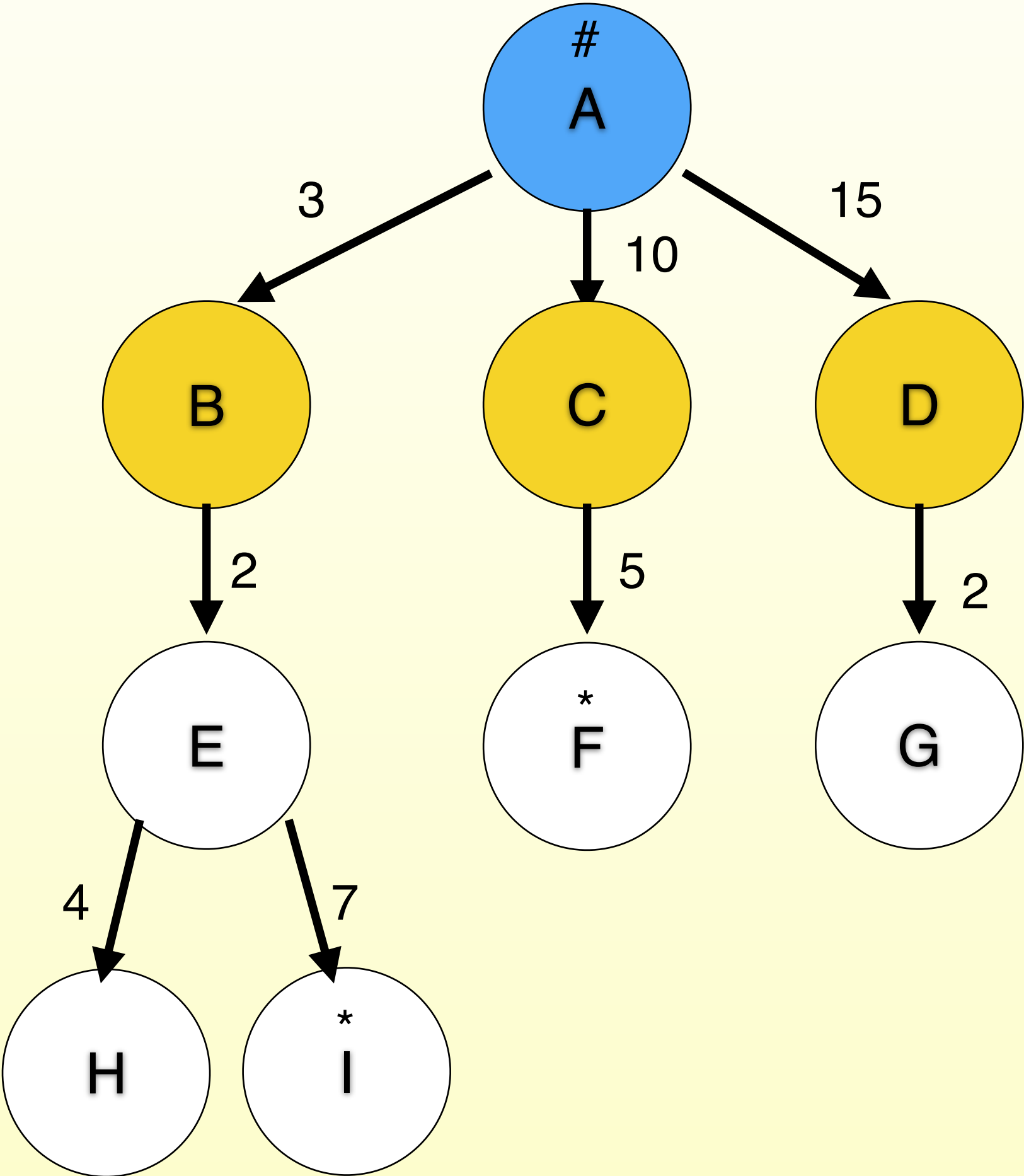
$\xrightarrow{c}$ Transition cost = c

Uniform Cost Search Example



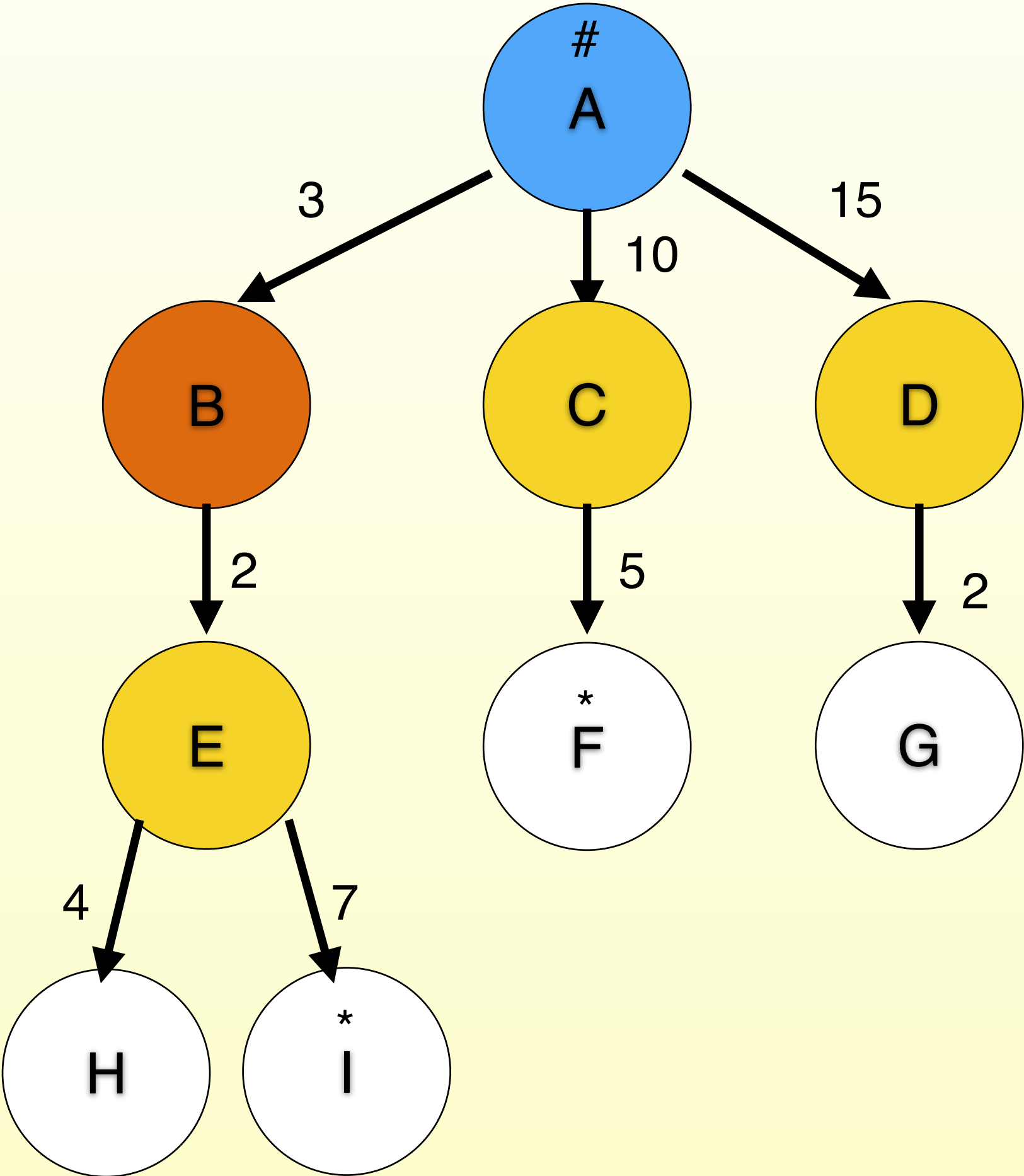
Node	g
A	0

Uniform Cost Search Example



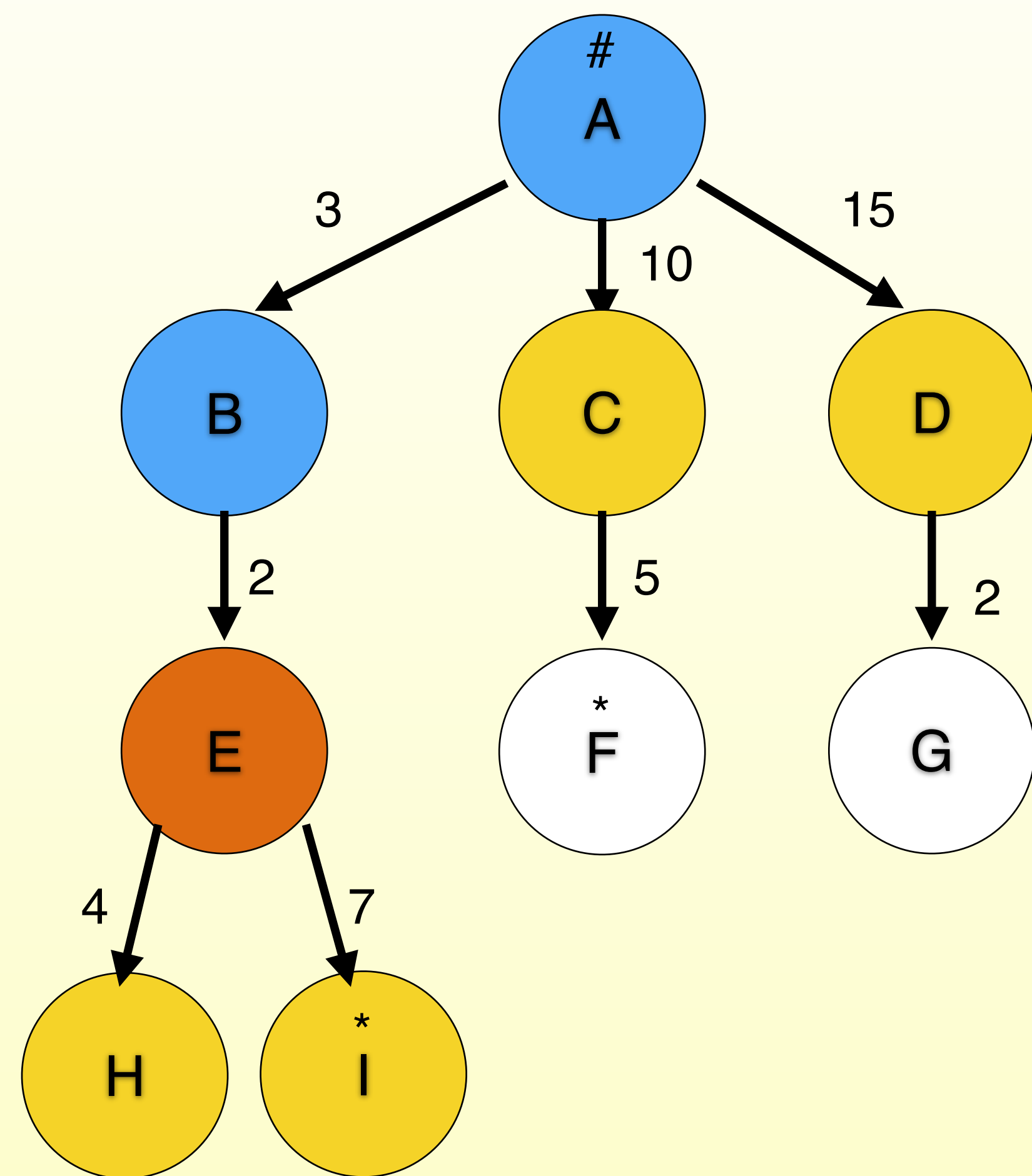
Node	g
B	3
C	10
D	15

Uniform Cost Search Example



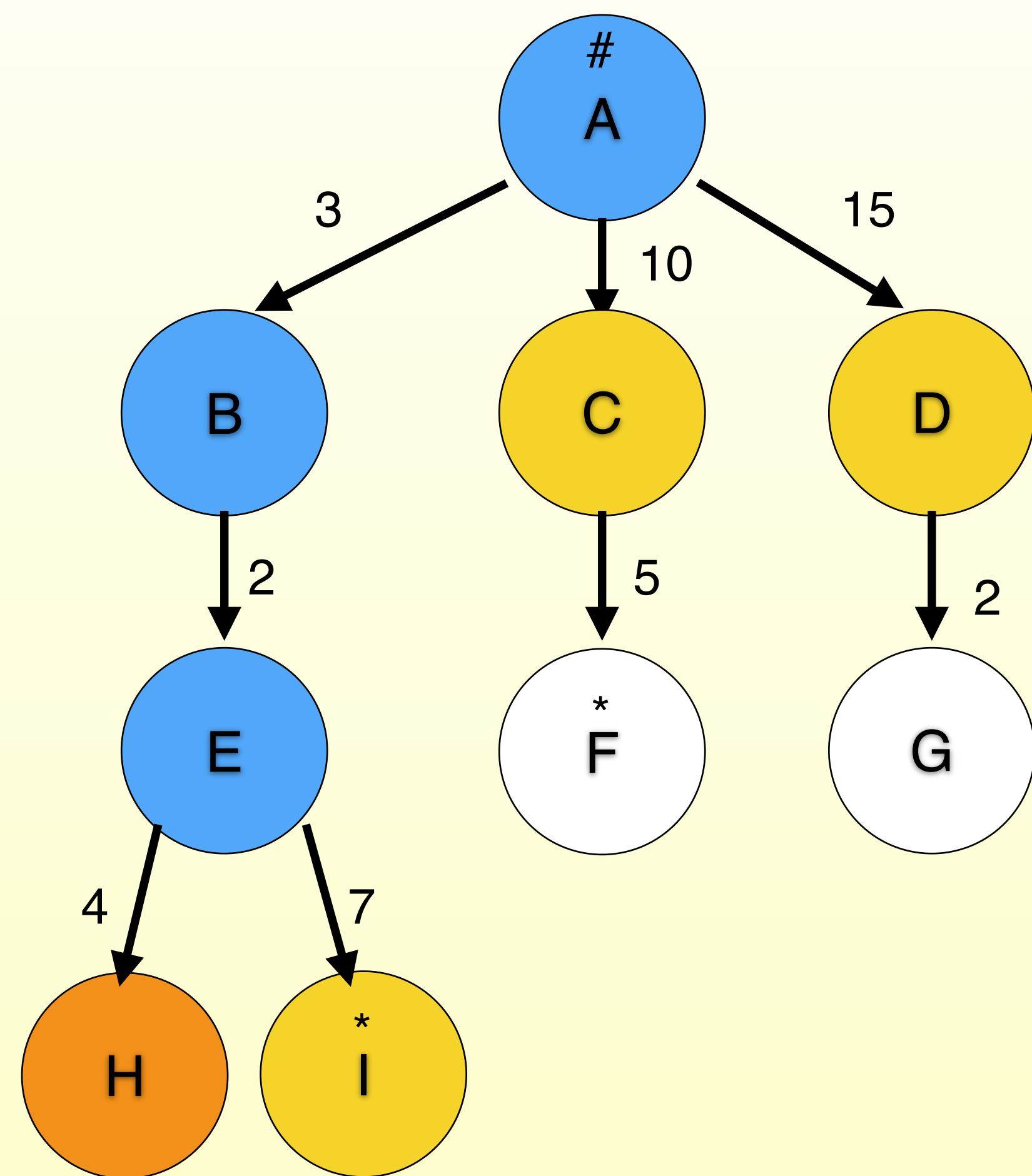
Node	g
E	3+2 = 5
C	10
D	15

Uniform Cost Search Example



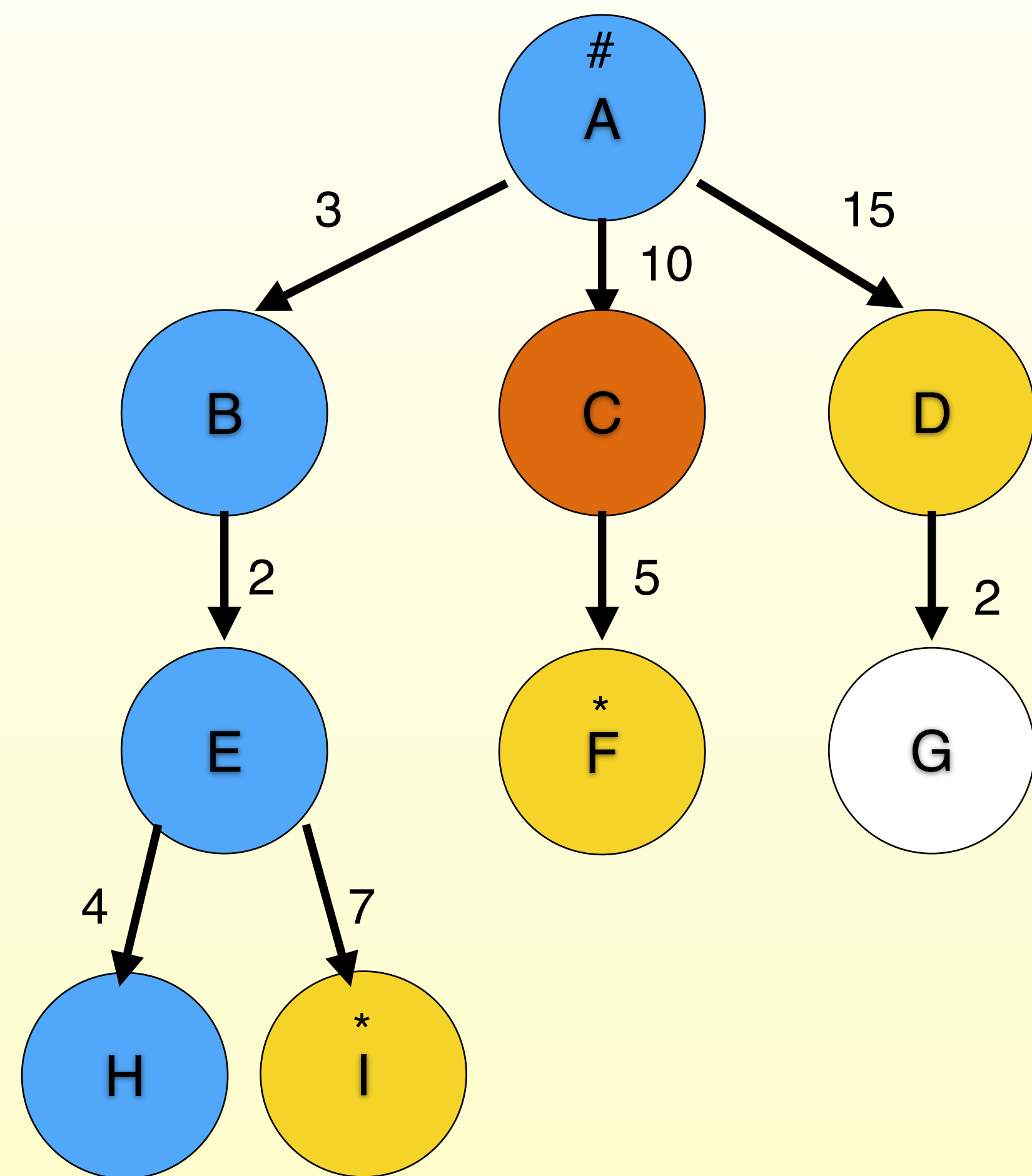
Node	g
H	$5+4 = 9$
C	10
I	$5+7=12$
D	15

Uniform Cost Search Example



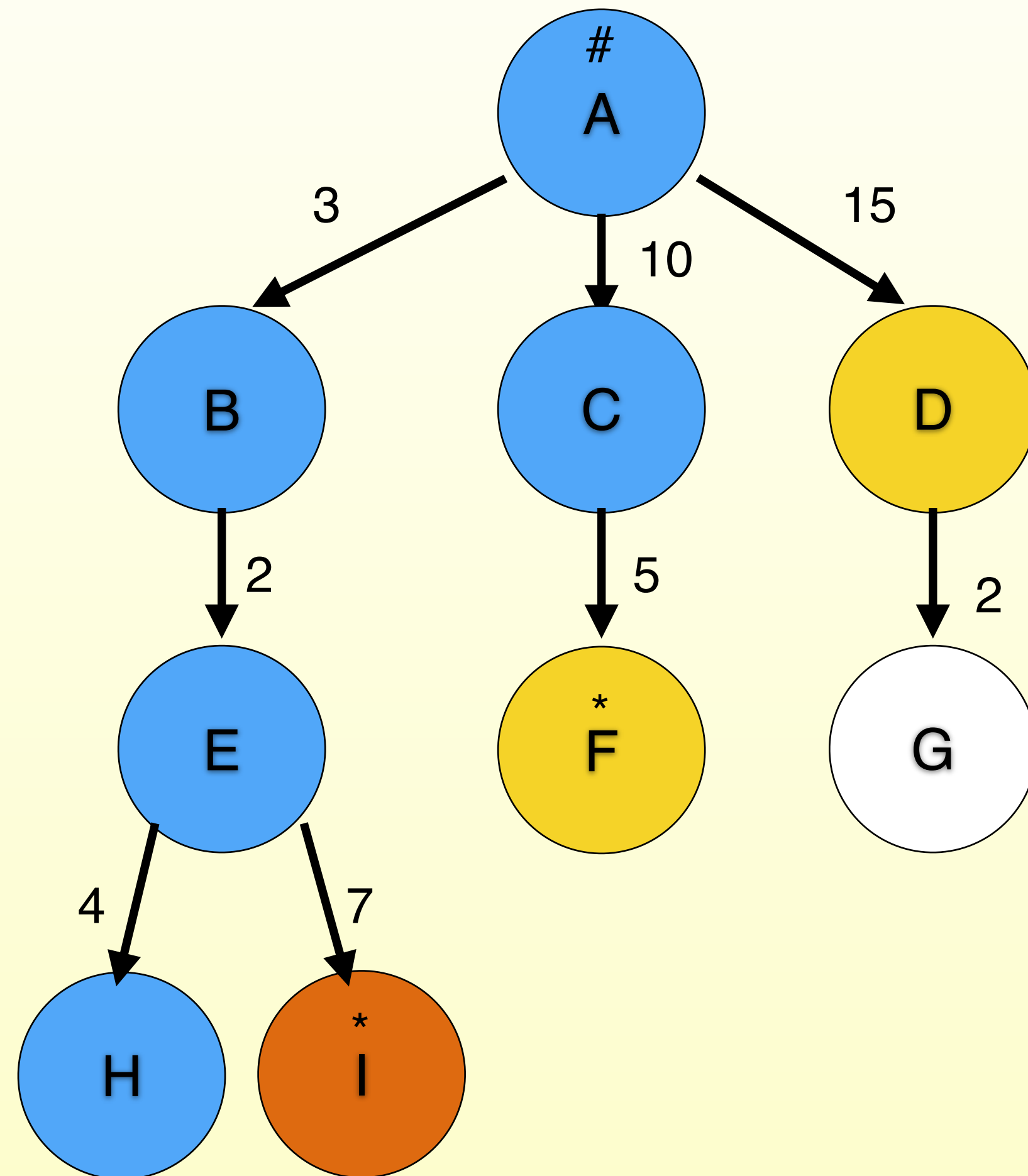
Node	g
H	5+4 = 9
C	10
I	5+7=12
D	15

Uniform Cost Search Example



Node	g
I	12
F	10+5=15
D	15

Uniform Cost Search Example



Goal found with cost 12!

Node	g
I	12
F	10+5=15
D	15

Uniform Cost Search vs other algorithms

- Uniform cost search can be seen as a variation of Dijkstra's Algorithm
- The typical formulation of Dijkstra is to find the shortest path from the root to **all nodes**
- In uniform cost, we typically stop once the first **goal node** is found
- DFS can be seen as a special case of uniform cost (where all edges have weight 1)
- Uniform cost can be seen as a special case of A^* (where $h=0$ for all nodes)
- See more at https://en.wikipedia.org/wiki/Dijkstra's_algorithm#Related_problems_and_algorithms

Minimum Spanning Tree

Definitions and properties

- A **tree** is any undirected graph that is both *connected* and *acyclic*
- Some properties:
 - A tree on n vertices has $n - 1$ edges
 - Any connected, undirected graph with $n - 1$ edges is a tree
 - In other words, a connected, undirected graph is a tree iff it has $n - 1$ edges
 - In a tree, there is a unique path between any given pair of vertices
 - If an undirected graph has a unique path between any pair of vertices, it is a tree
 - In other words, an undirected graph is a tree iff it has a unique path between any pair of vertices

Definitions and properties

- In other words, the following statements are all equivalent (for undirected graphs):
 - G is a Tree
 - G is connected and acyclic
 - G is connected and has $n-1$ edges
 - G has a unique path between any pair of vertices

Definitions and properties

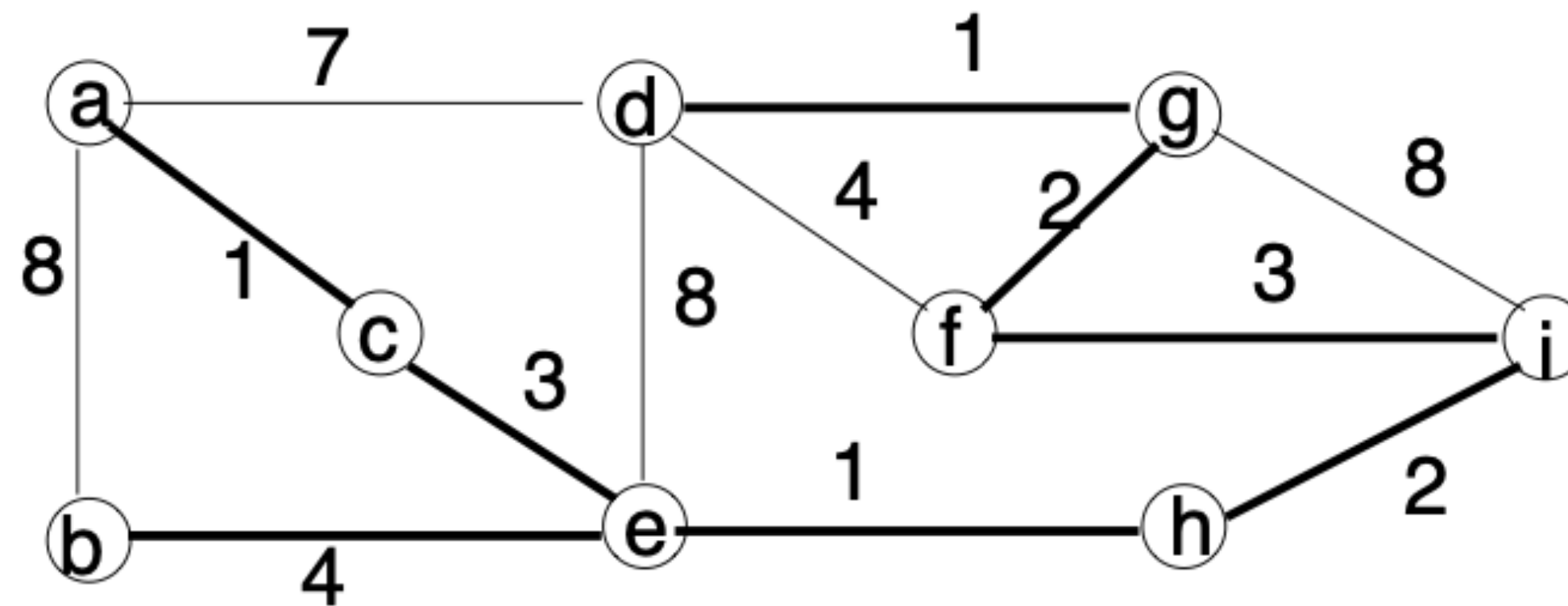
- Spanning tree: Given $G = \{V, E\}$, a subset $T \subseteq E$ is called a **Spanning Tree** if $G = \{V, T\}$ is connected and has no cycles
- Recall DFS: the edges identified as tree edges during search form a spanning tree
- What else can we use spanning trees for?

Minimum Spanning Tree

- Consider the following problem:
 - We have a network with n computers and m potential links between them
 - Each potential link has an associated cost (weight) if we choose to create it
 - We need to choose some links to maintain such that:
 - Any computer in the network can eventually reach any other computer via a series of links
 - The total maintenance cost of all links has to be as small as possible

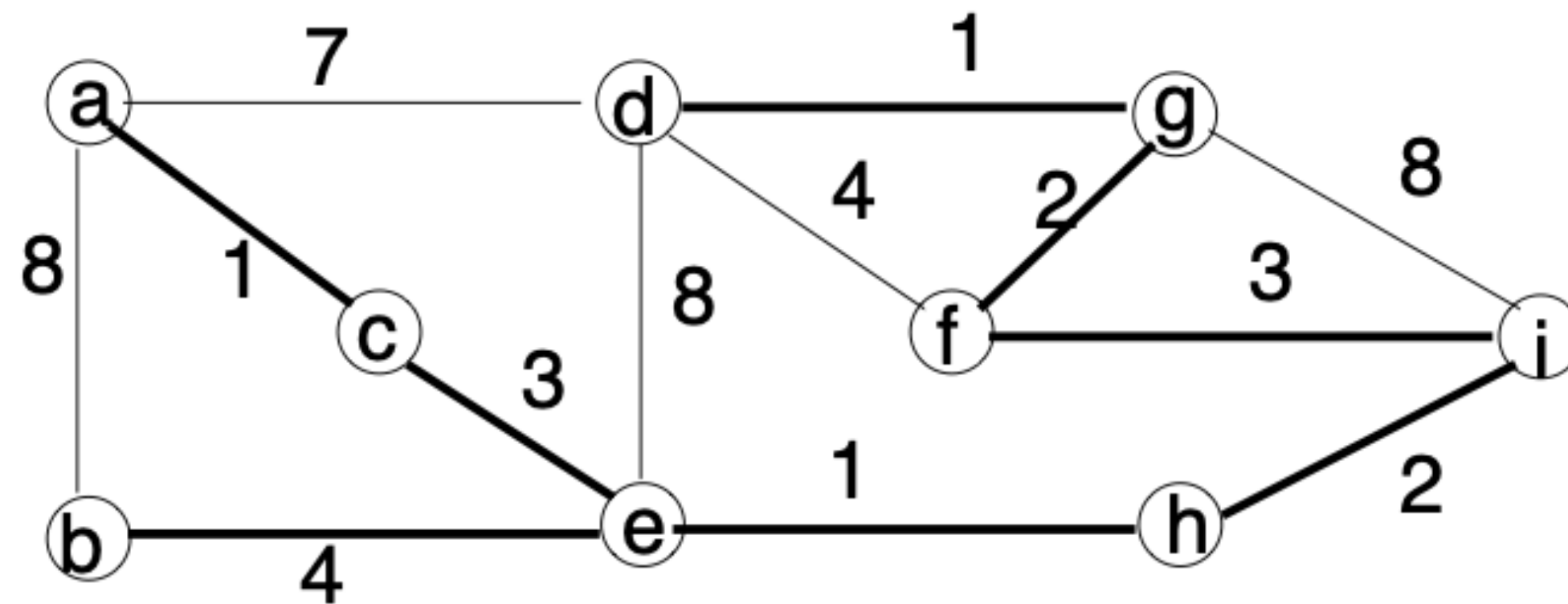
Minimum Spanning Tree

- Graph formulation: let $G = \{V, E\}$ be a weighted graph
 - V is the set of computers are vertices
 - E is the set of potential links, with an associated weight $w(e) > 0$ for each edge e
 - We need to choose a subset $S \subseteq E$ of edges such that
 - $G_S = \{V, S\}$ is connected
 - $\sum_{e \in S} w(e)$ is minimum



Minimum Spanning Tree

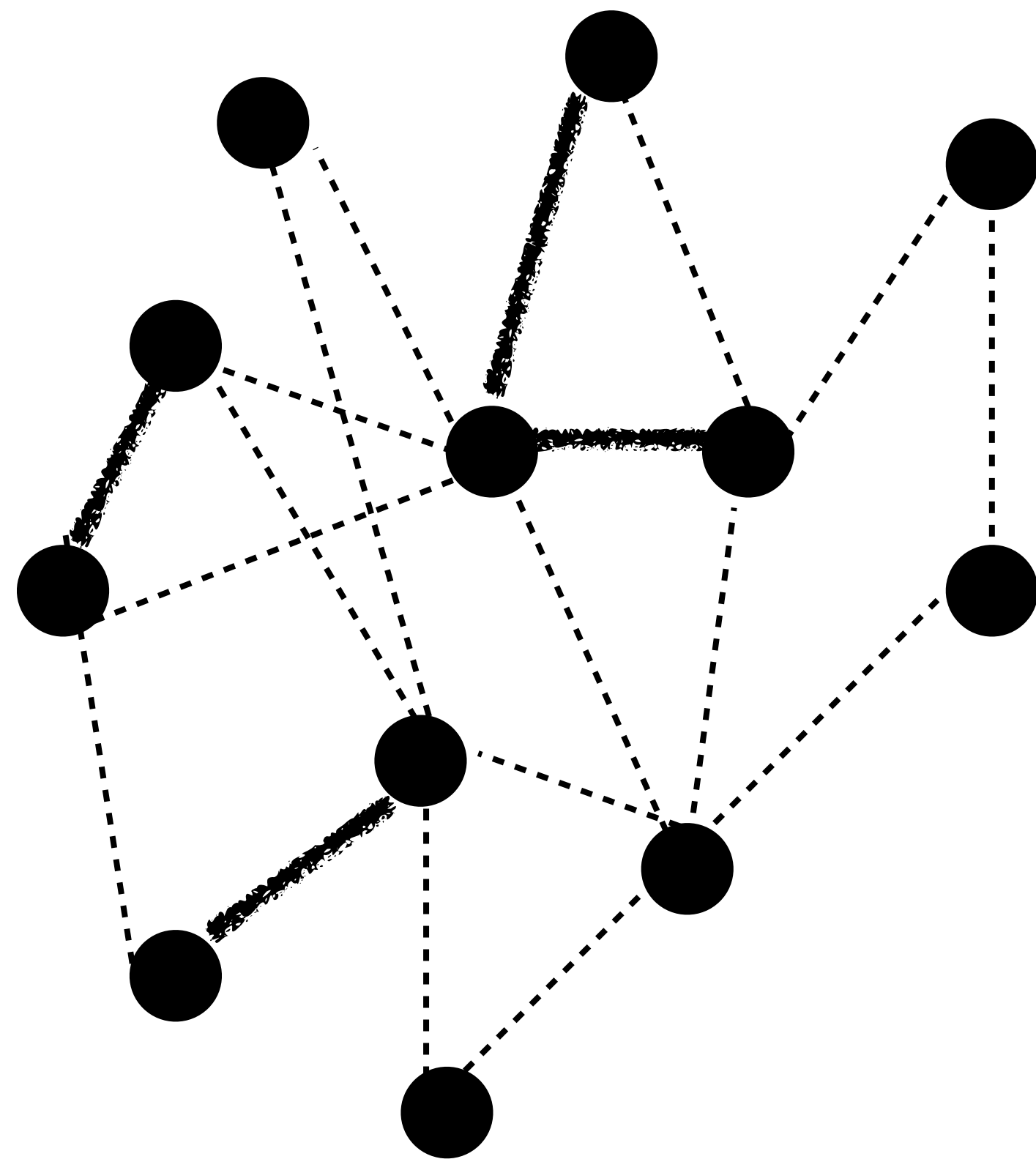
- Since we want to minimize the cost of selected edges, S should have no cycles. Therefore, G_S will be a tree
- S is a spanning tree, with minimum cost. A **Minimum Spanning Tree**
- A spanning tree for the example below is shown in bold



Cut property

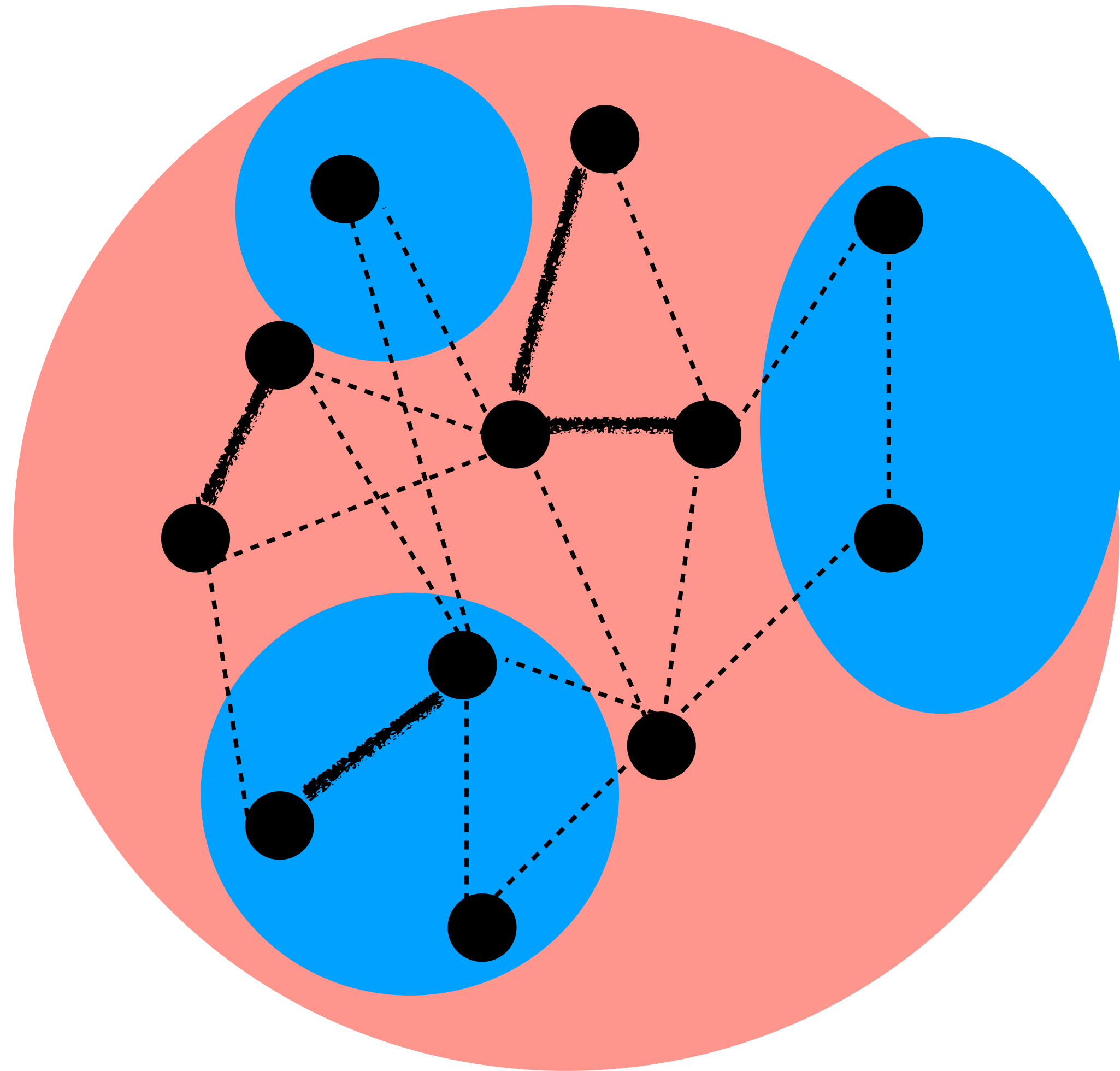
- Suppose we have $X \subseteq E$ that happens to be a subset of some minimum spanning tree T .
- Choose any subset $S \subseteq V$ such that X does not cross between S and $V - S$
 - We call this partition a cut
- *Lemma:* If e is the lightest edge across this cut, then $X \cup \{e\}$ must be part of some MST

A visual explanation



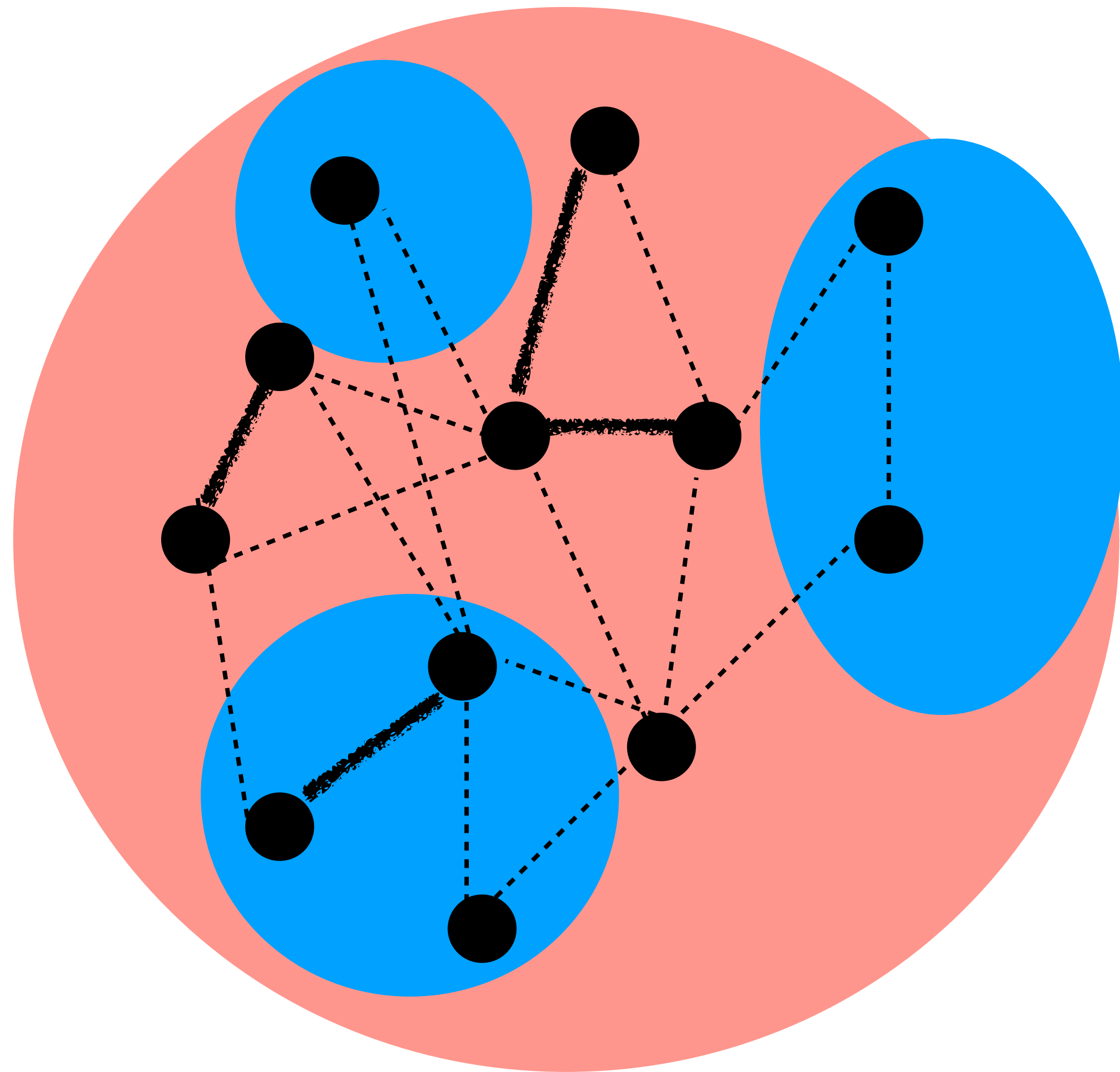
- Suppose you have a growing MST X .
- Highlight all edges of X in **bold**
- Represent the remaining edges as dashed lines

A visual explanation



- Divide your graph in two subregions
 - Vertices in Blue represent S
 - Vertices in Red (everything else) represents $V - S$
 - The regions do not need to be contiguous
 - The only rule is: no bold line may cross from Blue to Red
- This partition of vertices is a cut

A visual explanation



- The lemma:
- Pick any such partition
- The lightest dashed line that crosses regions is safe to add
- Exercise: highlight which dashed edges would be safe vs not safe if they had the smallest weight among dashed edges

Greedy Heuristic

- In other words: take any cut. If your growing MST X doesn't already cross the cut, then it is safe to add the lightest edge across it
- Idea: start with no edges, then iteratively add the lightest edge that is “safe” by selecting a suitable cut

```
ALGORITHM GENERIC_MST( $V, \text{Adj}, w$ )  
begin  
   $A \leftarrow \emptyset$ ;  
  while  $A$  is not a spanning tree do  
    find a safe edge  $(u, v) \in E$  to include in  $A$   
     $A = A \cup \{(u, v)\}$   
  endwhile  
  return  $A$ ;  
end
```

How to select a cut?

- A simple idea:
 - Start with any vertex as “Blue” and the rest of the graph as “Red”
 - The invariant: we will keep Blue as a single connected component containing all endpoints of X
 - Repeatedly add to X the lightest edge from the Blue component to the rest of the graph (Red)
- This strategy for choosing cuts gives us Prim’s algorithm

How to select a cut?

- Another idea
 - We will sort edges by weight initially, and maintain a list of disjoint connected components,
 - The invariant: No edge in our growing MST X crosses components
 - Initially, every vertex is its own component, and X is empty
 - Identify the lightest edge e between any two components C_i and C_j
 - Take the cut $C_i, V - C_i$
 - The MST doesn't cross this cut because no edge crosses from C_i to any other component (the invariant)
 - e crosses the cut by construction
 - Therefore e is safe to add to X and we merge the two components (maintaining the invariant)
- This gives us Kruskal's algorithm

Prim's Algorithm

- Maintains a single tree, initially with a single vertex
- Add a vertex to the tree by selecting the lightest edge between nodes already in and out of the current tree
- Requires a data structure that efficiently stores the cost of the smallest edge from a new node to the current tree - **priority queue**

Prim - Pseudocode

Algorithm 1 Prim's Algorithm

Input: A weighted, connected, undirected graph $G = (V, E)$

Output: Minimum Spanning Tree (MST) as a set of edges T

```
1:
2:  $T \leftarrow \emptyset$  // Initialize empty MST
3: Choose an arbitrary starting node  $s$ 
4: Initialize priority queue  $Q$  with  $(0, s)$  (priority = edge weight)
5: Initialize an empty set of visited nodes  $S$ 
6: Initialize parent array  $parent[v] \leftarrow \text{null}$  for all  $v$ 
7: while  $Q$  is not empty do
8:   Extract node  $u$  with minimum priority from  $Q$ 
9:   if  $u \notin S$  then
10:     Add  $u$  to  $S$ 
11:     if  $parent[u] \neq \text{null}$  then
12:       Add edge  $(parent[u], u)$  to  $T$ 
13:     for all neighbors  $v$  of  $u$  do
14:       if  $v \notin S$  then
15:         Insert  $(w(u, v), v)$  into  $Q$ 
16:         Set  $parent[v] \leftarrow u$ 
17: Return  $T$ 
```

Prim - Pseudocode

Algorithm 1 Prim's Algorithm

Input: A weighted, connected, undirected graph $G = (V, E)$

Output: Minimum Spanning Tree (MST) as a set of edges T

```
1:
2:  $T \leftarrow \emptyset$  // Initialize empty MST
3: Choose an arbitrary starting node  $s$ 
4: Initialize priority queue  $Q$  with  $(0, s)$  (priority = edge weight)
5: Initialize an empty set of visited nodes  $S$ 
6: Initialize parent array  $parent[v] \leftarrow \text{null}$  for all  $v$ 
7: while  $Q$  is not empty do
8:   Extract node  $u$  with minimum priority from  $Q$ 
9:   if  $u \notin S$  then
10:     Add  $u$  to  $S$ 
11:     if  $parent[u] \neq \text{null}$  then
12:       Add edge  $(parent[u], u)$  to  $T$ 
13:     for all neighbors  $v$  of  $u$  do
14:       if  $v \notin S$  then
15:         Insert  $(w(u, v), v)$  into  $Q$ 
16:         Set  $parent[v] \leftarrow u$ 
17: Return  $T$ 
```

Prim - Proof of correctness

- Proof by induction:
 - By the cut property, the lightest edge incident on the root the graph is always part of some MST (base case), and the subgraph containing only those two vertices is connected.
 - At each step we add a single edge to an expanding subtree X . Assume that at step k , X is part of some MST. Assume furthermore that X is connected (inductive hypothesis)
 - Let V_X be the set of all endpoints of any edge in X . Take the cut $\{V_X, V - V_X\}$
 - By construction, we select e as the lightest edge across this cut. Therefore, $X \cup e$ is part of some spanning tree.
 - By construction, $X \cup e$ is also connected
 - Therefore, after time step $k + 1$, X is connected and part of some MST
 - At time step $n - 1$, X has $n - 1$ edges and is connected. Therefore X is a tree of G . X is also part of some MST. Therefore, X is itself an MST

Prim - Variations

- There are several small variations possible, such as:
 - Storing edges rather than vertices in the heap
 - Storing all nodes or edges at the start rather than just the “root” (root does not grow dynamically)
 - Setting the root’s parent uniquely to itself
- These all have slight tradeoffs in terms of book-keeping and storage
- It is also common to represent the key, parent and visited arrays as properties of a node class or structure instead.

Prim - Priority Queue

- A popular implementation of a priority queue is as a binary heap
- A binary (min) heap is a complete binary tree with the property that every node stores a key less than or equal to its parent
 - Thus, the smallest key is always at the top
- A Binary heap can also be implemented as an implicit data structure, storing keys in a normal arrays and calculating child and parent positions with arithmetic operations over the indices
 - As a note, Python's default heap implementation, *heapq* **does** not provide a heap as an instantiable class, but as a collection of methods to manipulate a regular array according to this implicit data structure.
- Different types of heap have different time complexities, but for a binary heap, insertion is $O(1)$ on average, reading the min key is $O(1)$ in the worst case, but *removing* the min key is $O(\log(n))$

Prim - Time complexity

- Prim's algorithm performs up to $O(m)$ insertions and deletion operations on the heap
- Thus, its performance using binary heaps is $O(m * \log(m))$, or equivalently, $O(m * \log(n))$

Krushal's Algorithm

- Maintain a forest of smaller trees (components). Initially, each node is its own tree
- Initially sort all edges by weight
- At each step, join two trees by selecting the next smallest edge as long as it is across two different trees
- Requires an efficient way to test if two vertices belong to the same tree - **disjoint set** data structure

Kruskal's - Pseudocode

Algorithm 1 Kruskal's Minimum Spanning Tree Algorithm

```
1: Input: A weighted, connected, undirected graph  $G = (V, E)$ 
2: Output: Minimum Spanning Tree (MST) as a set of edges  $T$ 
3:
4:  $T \leftarrow \emptyset$  // Initialize empty MST
5: Sort all edges in  $E$  by increasing weight
6: Initialize Disjoint-Set data structure (Union-Find)
7: for all  $(u, v) \in E$  in sorted order do
8:   if Find( $u$ )  $\neq$  Find( $v$ ) then
9:     Add  $(u, v)$  to  $T$ 
10:    Union( $u, v$ )
11: Return  $T$ 
```

Krushal - Proof of correctness

- Proof by induction:
 - By the cut property, the lightest edge in the graph is always part of some MST (base case)
 - Assume that after looking at the k lightest edges, we have found some subset X of some MST. (induction hypothesis)
 - When we look at the next light edge $e_{k+1} = (u, v)$, there are two possibilities:
 - $find(u) = find(v)$ disjoint set. In this case, we do not modify X , so X remains a subset of some MST
 - $find(u) \neq find(v)$. Then u and v belong to different disjoint subsets S_1, S_2 . Take the cut $\{S_1, V - S_1\}$
 - None of the edges $e_1, e_2, \dots, e_k$ cross this cut, and e_{k+1} is the smallest remaining edge (and therefore the smallest across the cut).
 - By the cut property, $X \cup e_{k+1}$ is part of some MST
 - Therefore, after every step, X is part of some MST. At step m , there are no more edges to check, so (assuming G is connected), we have a single disjoint subset containing all the vertices. Thus X is an MST

Disjoint Sets data structures

- Key to Kruskal's algorithm is being able to efficiently check whether two nodes are in the same component
- Since no vertex can be in two components simultaneously, the components are disjoint
- A disjoint set data structure maintains each component as a disjoint set and provides operations to:
 - Merge two sets (Union)
 - Identify a unique and consistent "representative" for all members of a set (Find)
 - By checking if $\text{Find}(a) = \text{Find}(b)$, we can answer whether a and b are on the same set

Disjoint Sets - Implementation

- A common way to implement disjoint sets is as a **disjoint-set forest**
- While full specification of this structure is beyond our scope, each component is a rooted tree, with “parent” edges directed toward the root. The root can be uniquely identified by having no parent or having itself as parent
- The root thus serves as representative, and items in the same set will eventually find the same root
- Special care must be taken when merging to ensure that the tree remains of reasonable height
- Properly implemented, Union takes $O(1)$ time and Find takes only slightly worse than constant time (see https://en.wikipedia.org/wiki/Disjoint-set_data_structure#Time_complexity)

Kruskal - Complexity

- If G has n vertices and m edges, Kruskal's algorithm's complexity can thus be broken down as such:
 - Sorting all edges: $O(m * \log(m))$ which is equivalent (for connected graphs) to $O(m * \log(n))$
 - Iterating over all edges: we perform $O(m)$ Union and Find operations. This is very slightly slower than $O(m)$, and dominated by the sorting step
 - Thus, the algorithm runs in $O(m * \log(n))$